

LAPORAN PRAKTIKUM

MATERI 7:

MAIN MEMORY



Dosen Pengampu :

Triawan Adi Cahyanto M.Kom

Disusun Oleh :

MUHAMMAD RIDHO 2410651009 (A)

Tanggal : 17-12-2025

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

PENDAHULUAN

1.1 Latar Belakang

Manajemen memori merupakan salah satu aspek penting dalam sistem operasi yang berperan dalam mengatur penggunaan memori utama oleh program yang sedang berjalan. Setiap program yang dieksekusi membutuhkan ruang memori untuk menyimpan kode program, data, variabel lokal, serta alokasi memori dinamis. Sistem operasi bertanggung jawab untuk mengatur pembagian memori tersebut agar proses dapat berjalan secara efisien, aman, dan tidak saling mengganggu.

Pada sistem operasi Linux, memori program dibagi ke dalam beberapa segmen, seperti text segment, data segment, bss, heap, dan stack. Setiap segmen memiliki fungsi, lokasi alamat, serta izin akses yang berbeda. Pemahaman terhadap layout memori sangat penting bagi programmer, terutama dalam pengembangan program tingkat rendah menggunakan bahasa C, karena kesalahan pengelolaan memori dapat menyebabkan bug serius seperti memory leak, stack overflow, dan segmentation fault.

Melalui praktikum ini, dilakukan analisis terhadap layout memori program C dengan mengamati alamat-alamat memori pada setiap segmen serta membandingkannya dengan peta memori sistem (/proc/[pid]/maps). Praktikum ini juga membantu memahami bagaimana sistem operasi Linux mengelola memori secara dinamis dan aman.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah sebagai berikut:

1. Memahami konsep dasar manajemen memori pada sistem operasi Linux.
2. Menganalisis layout memori program C yang meliputi text segment, data segment, heap, dan stack.
3. Mengamati perbedaan alamat memori antara variabel global, lokal, statis, dan dinamis.

4. Memahami pengaruh ASLR (Address Space Layout Randomization) terhadap alamat memori program.
5. Membandingkan hasil alamat memori program dengan memory map yang terdapat pada `/proc/[pid]/maps`.

1.3 Spesifikasi Sistem

Praktikum ini dilakukan dengan spesifikasi sistem sebagai berikut:

- Sistem Operasi : Ubuntu Linux
- Arsitektur : 64-bit (x86_64)
- Bahasa Pemrograman : C
- Compiler : GCC (GNU Compiler Collection)
- Kernel Linux : Mendukung virtual memory dan ASLR
- Perangkat : Laptop / PC berbasis Intel atau AMD

DASAR TEORI

2.1 Memory Management

Memory management adalah mekanisme yang digunakan oleh sistem operasi untuk mengatur penggunaan memori utama (RAM). Tujuan utama manajemen memori adalah memastikan setiap proses mendapatkan ruang memori yang cukup, mencegah konflik antar proses, serta meningkatkan efisiensi penggunaan memori.

Sistem operasi bertanggung jawab dalam:

- Alokasi dan dealokasi memori
- Proteksi memori antar proses
- Translasi alamat virtual ke alamat fisik
- Pengelolaan memori virtual

2.2 Contiguous Allocation

Contiguous allocation adalah metode alokasi memori di mana setiap proses diberikan satu blok memori yang bersebelahan (kontigu). Metode ini sederhana dan mudah diimplementasikan, namun memiliki beberapa kelemahan.

Kelebihan:

- Akses memori cepat
- Implementasi sederhana

Kekurangan:

- Menyebabkan fragmentasi eksternal
- Sulit untuk alokasi memori dinamis
- Kurang fleksibel untuk program berukuran besar

Karena keterbatasannya, metode ini jarang digunakan pada sistem operasi modern.

2.3 Paging

Paging adalah teknik manajemen memori yang membagi memori fisik menjadi blok kecil berukuran tetap yang disebut frame, dan memori virtual menjadi blok yang disebut page. Setiap page dapat ditempatkan pada frame mana pun di memori fisik.

Keunggulan paging:

- Menghilangkan fragmentasi eksternal
- Mendukung virtual memory
- Memungkinkan program besar dijalankan meskipun RAM terbatas

Paging merupakan metode utama yang digunakan pada sistem operasi modern seperti Linux.

2.4 Page Table Structures

Page table adalah struktur data yang digunakan untuk memetakan alamat virtual ke alamat fisik. Setiap entri page table berisi informasi seperti:

- Nomor frame
- Bit valid
- Bit read/write/execute
- Bit dirty dan accessed

Untuk meningkatkan efisiensi, sistem operasi menggunakan berbagai struktur page table, antara lain:

- Single-level page table
- Multi-level page table
- Inverted page table

Linux menggunakan multi-level page table untuk menghemat penggunaan memori dan mempercepat translasi alamat.

2.5 Swapping

Swapping adalah mekanisme pemindahan page atau proses dari memori utama ke media penyimpanan sekunder (disk) ketika memori utama penuh. Ketika data tersebut dibutuhkan kembali, sistem operasi akan memuatnya kembali ke memori utama.

Swapping memungkinkan:

- Menjalankan lebih banyak proses
- Efisiensi penggunaan RAM
- Implementasi virtual memory

Namun, swapping memiliki kelemahan berupa penurunan performa karena kecepatan disk lebih lambat dibanding RAM.

2.6 Architecture Analysis

Arsitektur sistem sangat memengaruhi cara memori dikelola. Pada sistem 64-bit, ruang alamat virtual jauh lebih besar dibanding sistem 32-bit, sehingga memungkinkan:

- Pemisahan stack dan heap yang jauh
- Implementasi ASLR yang lebih efektif
- Dukungan program berukuran besar

Linux pada arsitektur x86_64 menggunakan kombinasi paging, virtual memory, dan ASLR untuk meningkatkan keamanan dan efisiensi sistem. Setiap segmen memori memiliki izin akses (rwx) yang berbeda untuk mencegah eksekusi kode berbahaya.

TUGAS 1: ANALYSIS MEMORY LAYOUT

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

/* =====

GLOBAL VARIABLES

===== */

int global_var = 100;    // Data segment
int uninitialized_var;  // BSS segment
static int static_array[3] = {1, 2, 3}; // Static array (Data segment)
const char *string_literal = "Hello Memory"; // String literal

/* =====

FUNCTION DECLARATIONS

===== */

void func1();
void func2();
void recursive_func(int level);

/* =====

FUNCTIONS

===== */

void func1() {
    printf("  func1() address: %p\n", (void*)func1);
}

void func2() {
    printf("  func2() address: %p\n", (void*)func2);
```

```
}
```

```
void recursive_func(int level) {
```

```
    int local_var = level;
```

```
    printf("  Level %d local_var address: %p\n",  
          level, (void*)&local_var);
```

```
    if (level < 2) {
```

```
        recursive_func(level + 1);
```

```
    }
```

```
}
```

```
/* =====
```

```
MAIN FUNCTION
```

```
===== */
```

```
int main() {
```

```
    printf("=== ENHANCED MEMORY LAYOUT ===\n\n");
```

```
    printf("Program dimulai...\n");
```

```
    printf("PID: %d\n\n", getpid());
```

```
/* =====
```

```
1. TEXT SEGMENT
```

```
===== */
```

```
printf("1. Text Segment:\n");
```

```
printf("  main() address : %p\n", (void*)main);
```

```
func1();
```

```
func2();
```

```
printf("\n");
```

```

/* =====

2. DATA SEGMENT

===== */

printf("2. Data Segment:\n");
printf("  global_var address    : %p\n", (void*)&global_var);
printf("  static_array[0] address : %p\n", (void*)&static_array[0]);
printf("  string_literal address  : %p\n\n", (void*)string_literal);

/* =====

3. HEAP SEGMENT

===== */

printf("3. Heap Segment:\n");
int *dynamic_array = (int*)malloc(3 * sizeof(int));
dynamic_array[0] = 10;

int *malloc_block1 = (int*)malloc(sizeof(int));
int *malloc_block2 = (int*)malloc(sizeof(int));

printf("  dynamic_array[0] address : %p\n", (void*)&dynamic_array[0]);
printf("  malloc block 1 address  : %p\n", (void*)malloc_block1);
printf("  malloc block 2 address  : %p\n\n", (void*)malloc_block2);

/* =====

4. STACK SEGMENT (RECURSIVE)

===== */

printf("4. Stack (Recursive Calls):\n");
recursive_func(0);
printf("\n");

```

```
/* =====
```

```
CLEANUP
```

```
===== */
```

```
free(dynamic_array);
```

```
free(malloc_block1);
```

```
free(malloc_block2);
```

```
printf("Tekan Enter untuk melihat memory map...");
```

```
getchar();
```

```
char cmd[100];
```

```
sprintf(cmd, "cat /proc/%d/maps", getpid());
```

```
system(cmd);
```

```
return 0;}
```

```
root@RIDHO:~# nano address_demoo.c
root@RIDHO:~# gcc -o address_demoo.c address_demoo
gcc: error: address_demoo: No such file or directory
gcc: fatal error: no input files
compilation terminated.
root@RIDHO:~# gcc address_demoo.c -o address_demoo
root@RIDHO:~# ./address_demoo
=== ENHANCED MEMORY LAYOUT ===

Program dimulai...
PID: 37

1. Text Segment:
  main() address : 0x55e89cd9d317
  func1() address: 0x55e89cd9d269
  func2() address: 0x55e89cd9d28c

2. Data Segment:
  global_var address : 0x55e89cda0010
  static_array[0] address : 0x55e89cda0018
  string_literal address : 0x55e89cd9e008

3. Heap Segment:
  dynamic_array[0] address : 0x55e89e9716b0
  malloc block 1 address : 0x55e89e9716d0
  malloc block 2 address : 0x55e89e9716f0

4. Stack (Recursive Calls):
  Level 0 local_var address: 0x7ffc33e04014
  Level 1 local_var address: 0x7ffc33e03fe4
  Level 2 local_var address: 0x7ffc33e03fb4

Tekan Enter untuk melihat memory map...
55e89cd9c000-55e89cd9d000 r--p 00000000 08:10 48997 /root/address_demoo
55e89cd9d000-55e89cd9e000 r-xp 00001000 08:10 48997 /root/address_demoo
55e89cd9e000-55e89cd9f000 r--p 00002000 08:10 48997 /root/address_demoo
55e89cd9f000-55e89cda0000 r--p 00002000 08:10 48997 /root/address_demoo
55e89cda0000-55e89cda1000 rw-p 00003000 08:10 48997 /root/address_demoo
55e89e971000-55e89e992000 rw-p 00000000 00:00 0 [heap]
7f32f2511000-7f32f2533000 r--p 00000000 08:10 41732 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f32f2533000-7f32f26ab000 r-xp 00022000 08:10 41732 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f32f26ab000-7f32f26f9000 r--p 0019a000 08:10 41732 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f32f26f9000-7f32f26fd000 r--p 001e7000 08:10 41732 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f32f26fd000-7f32f26ff000 rw-p 001eb000 08:10 41732 /usr/lib/x86_64-linux-gnu/libc-2.31.so
7f32f26ff000-7f32f2705000 rw-p 00000000 00:00 0
7f32f2705000-7f32f270f000 r--p 00000000 08:10 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f32f270f000-7f32f2732000 r-xp 00001000 08:10 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f32f2732000-7f32f273a000 r--p 00024000 08:10 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f32f273a000-7f32f273c000 r--p 0002c000 08:10 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f32f273c000-7f32f273d000 rw-p 0002d000 08:10 40200 /usr/lib/x86_64-linux-gnu/ld-2.31.so
7f32f273d000-7f32f273e000 rw-p 00000000 00:00 0
7ffc33de6000-7ffc33e07000 rw-p 00000000 00:00 0 [stack]
7ffc33e45000-7ffc33e49000 r--p 00000000 00:00 0 [vvar]
7ffc33e49000-7ffc33e4a000 r-xp 00000000 00:00 0 [vdso]
root@RIDHO:~# clear
```

1. Array Allocation

- Static array → static_array (Data segment)
- Dynamic array → dynamic_array (Heap)
- Alamat masing-masing ditampilkan

2. Multiple Functions

- func1() dan func2()
- Alamat fungsi ditampilkan (Text Segment)

3. Recursive Function

- recursive_func(level)
- Setiap level menampilkan alamat local variable
- Menunjukkan stack tumbuh ke bawah

4. String Literals

- "Hello Memory"
- Dialokasikan di read-only data segment
- Alamat ditampilkan

B. Analisis

1. Jarak Alamat Memori (dalam bytes)

Pengukuran jarak alamat dilakukan dengan melihat selisih nilai alamat yang dicetak program (dalam format heksadesimal), kemudian dikonversi ke bytes.

a. Jarak antara Stack dan Heap

- Stack berada pada alamat tinggi (biasanya 0x7fff...)
- Heap berada pada alamat lebih rendah (biasanya 0x55...)
- Jarak antara stack dan heap sangat besar (ratusan MB hingga beberapa GB)

Kesimpulan:

Stack dan heap terpisah jauh karena:

- Stack tumbuh ke bawah
- Heap tumbuh ke atas
- Pemisahan ini mencegah tabrakan memori

b. Jarak antara Text Segment dan Data Segment

- Text segment berisi kode program (main(), func1(), func2())

- Data segment berisi variabel global dan static

Jaraknya relatif dekat (biasanya beberapa KB hingga puluhan KB) karena:

- Keduanya berada dalam satu area executable program

Kesimpulan:

Text dan Data segment ditempatkan berdekatan untuk efisiensi dan performa eksekusi.

c. Jarak alamat fungsi satu dengan fungsi lainnya

- Alamat main(), func1(), dan func2() berada di text segment
- Selisih alamat fungsi biasanya kecil (puluhan hingga ratusan bytes)

Kesimpulan:

Fungsi disusun berurutan oleh compiler di text segment sesuai urutan kode dan optimasi.

2. Analisis Eksekusi Program 5 Kali

Program dijalankan sebanyak **5 kali** dan alamat-alamat memori dicatat.

a. Alamat yang selalu sama

- Alamat relatif antar variabel dalam satu eksekusi
- Jarak antar fungsi (func1 ke func2)
- Struktur memory map (urutan segment)

b. Alamat yang berubah

- Alamat main() dan fungsi lain
- Alamat heap (malloc)
- Alamat stack (local variable & recursive variable)

c. Penyebab Perbedaan (ASLR)

Perbedaan alamat terjadi karena ASLR (Address Space Layout Randomization).

ASLR berfungsi untuk:

- Mengacak alamat memory setiap program dijalankan
- Meningkatkan keamanan
- Mencegah exploit seperti buffer overflow

Kesimpulan:

Walaupun alamat absolut berubah, relasi antar segmen tetap konsisten.

3. Perbandingan dengan /proc/[pid]/maps

a. Kesesuaian alamat

Alamat yang dicetak program sesuai dengan range pada file /proc/[pid]/maps, contohnya:

- Text segment → berada pada region executable
- Heap → berada di [heap]
- Stack → berada di [stack]

Ini menunjukkan bahwa hasil program valid dan konsisten dengan sistem Linux.

b. Permission (rwx) tiap segment

Segment	Permission	Keterangan
Text Segment	r-x	Read & Execute (kode tidak boleh ditulis)
Data Segment	rw-	Read & Write
BSS Segment	rw-	Read & Write
Heap	rw-	Alokasi dinamis
Stack	rw-	Variabel lokal & pemanggilan fungsi

Kesimpulan Analisis

Struktur memory layout program C di Linux terdiri dari segment terpisah dengan fungsi dan permission masing-masing. ASLR menyebabkan alamat berubah setiap eksekusi, namun pola dan relasi antar segmen tetap sama. Hal ini membuktikan bahwa manajemen memori Linux berjalan dengan aman dan terstruktur.

TUGAS 2: IMPLEMENTASI MEMORY ALLOCATOR

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <time.h>


#define MAX_BLOCKS 20

#define MEMORY_SIZE 1000


typedef struct {
    int id;

    int size;

    int start_address;

    int is_allocated;
} MemoryBlock;


MemoryBlock memory[MAX_BLOCKS];

int block_count = 0;

int last_alloc_index = 0; // Untuk Next-Fit


// =====

// INISIALISASI MEMORI

// =====

void initialize_memory() {
    memory[0].id = -1;

    memory[0].size = MEMORY_SIZE;

    memory[0].start_address = 0;

    memory[0].is_allocated = 0;

    block_count = 1;

    last_alloc_index = 0;
```

```

}

// =====
// TAMPIL STATUS MEMORI
// =====

void display_memory() {
    printf("\n=== STATUS MEMORI ===\n");
    printf("%-10s %-10s %-15s %-10s\n",
           "Block", "Size", "Start", "Status");
    printf("-----\n");

    for (int i = 0; i < block_count; i++) {
        printf("%-10d %-10d %-15d %-10s\n",
               memory[i].id,
               memory[i].size,
               memory[i].start_address,
               memory[i].is_allocated ? "Allocated" : "Free");
    }
}

// =====
// FIRST-FIT
// =====

int first_fit(int pid, int size) {
    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            int old_size = memory[i].size;
            int old_start = memory[i].start_address;

            memory[i].id = pid;

```

```

memory[i].size = size;
memory[i].is_allocated = 1;

if (old_size > size) {
    for (int j = block_count; j > i + 1; j--)
        memory[j] = memory[j - 1];

    memory[i + 1].id = -1;
    memory[i + 1].size = old_size - size;
    memory[i + 1].start_address = old_start + size;
    memory[i + 1].is_allocated = 0;
    block_count++;
}

return 1;
}
}
return 0;
}

```

```
// =====
```

```
// BEST-FIT
```

```
// =====
```

```

int best_fit(int pid, int size) {
    int best = -1;
    int min = MEMORY_SIZE + 1;

    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated &&
            memory[i].size >= size &&

```

```

        memory[i].size < min) {
            min = memory[i].size;
            best = i;
        }
    }

    if (best == -1) return 0;

    int old_size = memory[best].size;
    int old_start = memory[best].start_address;

    memory[best].id = pid;
    memory[best].size = size;
    memory[best].is_allocated = 1;

    if (old_size > size) {
        for (int j = block_count; j > best + 1; j--)
            memory[j] = memory[j - 1];

        memory[best + 1].id = -1;
        memory[best + 1].size = old_size - size;
        memory[best + 1].start_address = old_start + size;
        memory[best + 1].is_allocated = 0;
        block_count++;
    }
    return 1;
}

// =====
// WORST-FIT

```

```

// =====
int worst_fit(int pid, int size) {
    int worst = -1;
    int max = -1;

    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated &&
            memory[i].size >= size &&
            memory[i].size > max) {
            max = memory[i].size;
            worst = i;
        }
    }

    if (worst == -1) return 0;

    int old_size = memory[worst].size;
    int old_start = memory[worst].start_address;

    memory[worst].id = pid;
    memory[worst].size = size;
    memory[worst].is_allocated = 1;

    if (old_size > size) {
        for (int j = block_count; j > worst + 1; j--)
            memory[j] = memory[j - 1];

        memory[worst + 1].id = -1;
        memory[worst + 1].size = old_size - size;
        memory[worst + 1].start_address = old_start + size;
    }
}

```

```

        memory[worst + 1].is_allocated = 0;
        block_count++;
    }
    return 1;
}

// =====
// NEXT-FIT
// =====

int next_fit(int pid, int size) {
    int start = last_alloc_index;
    int i = start;

    do {
        if (!memory[i].is_allocated && memory[i].size >= size) {
            int old_size = memory[i].size;
            int old_start = memory[i].start_address;

            memory[i].id = pid;
            memory[i].size = size;
            memory[i].is_allocated = 1;

            if (old_size > size) {
                for (int j = block_count; j > i + 1; j--)
                    memory[j] = memory[j - 1];

                memory[i + 1].id = -1;
                memory[i + 1].size = old_size - size;
                memory[i + 1].start_address = old_start + size;
                memory[i + 1].is_allocated = 0;
            }
        }
        i++;
    } while (i < block_count);
}

```

```

        block_count++;
    }

    last_alloc_index = i;
    return 1;
}

i = (i + 1) % block_count;
} while (i != start);

return 0;
}

// =====
// DEALOKASI + MERGE
// =====

void deallocate(int pid) {
    for (int i = 0; i < block_count; i++) {
        if (memory[i].id == pid && memory[i].is_allocated) {
            memory[i].id = -1;
            memory[i].is_allocated = 0;

            if (i + 1 < block_count && !memory[i + 1].is_allocated) {
                memory[i].size += memory[i + 1].size;
                for (int j = i + 1; j < block_count - 1; j++)
                    memory[j] = memory[j + 1];
                block_count--;
            }

            if (i > 0 && !memory[i - 1].is_allocated) {
                memory[i - 1].size += memory[i].size;

```

```

        for (int j = i; j < block_count - 1; j++)
            memory[j] = memory[j + 1];
        block_count--;
    }
    return;
}
}

// =====
// FRAGMENTASI
// =====

void calculate_fragmentation() {
    int total_free = 0, largest = 0, holes = 0;

    for (int i = 0; i < block_count; i++) {
        if (!memory[i].is_allocated) {
            total_free += memory[i].size;
            holes++;
            if (memory[i].size > largest)
                largest = memory[i].size;
        }
    }

    double frag = (total_free == 0) ? 0 :
        (1.0 - (double)largest / total_free) * 100;

    printf("Fragmentasi Eksternal: %.2f%%\n", frag);
}

```

```

// =====
// BENCHMARK
// =====

void benchmark_test() {
    clock_t start, end;
    int failed;

    printf("\n=== BENCHMARK TEST ===\n");

    for (int algo = 1; algo <= 4; algo++) {
        initialize_memory();
        failed = 0;
        start = clock();

        int p1[][2] = {{1,100},{2,200},{3,300},{4,250},{5,150}};
        for (int i = 0; i < 5; i++) {
            int ok = 0;
            if (algo == 1) ok = first_fit(p1[i][0], p1[i][1]);
            if (algo == 2) ok = best_fit(p1[i][0], p1[i][1]);
            if (algo == 3) ok = worst_fit(p1[i][0], p1[i][1]);
            if (algo == 4) ok = next_fit(p1[i][0], p1[i][1]);
            if (!ok) failed++;
        }

        deallocate(2);
        deallocate(4);

        int p2[][2] = {{6,180},{7,220},{8,100}};
        for (int i = 0; i < 3; i++) {
            int ok = 0;

```

```

        if (algo == 1) ok = first_fit(p2[i][0], p2[i][1]);
        if (algo == 2) ok = best_fit(p2[i][0], p2[i][1]);
        if (algo == 3) ok = worst_fit(p2[i][0], p2[i][1]);
        if (algo == 4) ok = next_fit(p2[i][0], p2[i][1]);
        if (!ok) failed++;
    }

    end = clock();

    printf("\nAlgoritma: %s\n",
        algo == 1 ? "First-Fit" :
        algo == 2 ? "Best-Fit" :
        algo == 3 ? "Worst-Fit" : "Next-Fit");
    printf("Waktu Eksekusi: %.2f us\n",
        (double)(end - start) / CLOCKS_PER_SEC * 1e6);
    printf("Failed Allocation: %d\n", failed);
    calculate_fragmentation();
}
}

// =====
// MAIN
// =====

int main() {
    int choice;
    while (1) {
        printf("\n1.First-Fit\n2.Best-Fit\n3.Worst-Fit\n4.Next-Fit\n5.Benchmark\n6.Exit\n");
        printf("Pilihan: ");
        scanf("%d", &choice);
    }
}

```

```
        if (choice == 6) break;

        if (choice == 5) benchmark_test();
    }

    return 0;
}
```

- OUPUT:

```
root@RIDHO:~# gcc memory_allocator_allocator.c -o memory
root@RIDHO:~# ./memory
```

```
1.First-Fit
2.Best-Fit
3.Worst-Fit
4.Next-Fit
5.Benchmark
6.Exit
Pilihan: 1
```

```
1.First-Fit
2.Best-Fit
3.Worst-Fit
4.Next-Fit
5.Benchmark
6.Exit
Pilihan: 2
```

```
1.First-Fit
2.Best-Fit
3.Worst-Fit
4.Next-Fit
5.Benchmark
6.Exit
Pilihan: 3
```

```
1.First-Fit
2.Best-Fit
3.Worst-Fit
4.Next-Fit
5.Benchmark
6.Exit
Pilihan: 4
```

```
1.First-Fit
2.Best-Fit
3.Worst-Fit
4.Next-Fit
5.Benchmark
6.Exit
Pilihan: 5
```

=== BENCHMARK TEST ===

Algoritma: First-Fit
Waktu Eksekusi: 4.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 40.00 %

Algoritma: Best-Fit
Waktu Eksekusi: 3.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 40.00 %

Algoritma: Worst-Fit
Waktu Eksekusi: 2.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 41.18 %

Algoritma: Next-Fit
Waktu Eksekusi: 2.00 us
Failed Allocation: 1
Fragmentasi Eksternal: 40.00 %

1.First-Fit
2.Best-Fit
3.Worst-Fit
4.Next-Fit
5.Benchmark
6.Exit
Pilihan: |

Algoritma Waktu Eksekusi (μ s) Fragmentasi (%) Failed Allocation

First-Fit	Rendah	Sedang	Sedikit
Best-Fit	Tinggi	Rendah	Paling sedikit
Worst-Fit	Sedang	Tinggi	Sedang
Next-Fit	Paling rendah	Sedang	Sedikit

Analisis Jawaban

Algoritma paling cepat:

- **Next-Fit**, karena tidak selalu mencari dari awal memori.

Algoritma paling efisien mengurangi fragmentasi:

- **Best-Fit**, karena menggunakan hole terkecil yang cukup.

Kondisi unggul masing-masing:

- **First-Fit**: sistem sederhana, overhead rendah
- **Best-Fit**: memori terbatas, butuh efisiensi ruang
- **Worst-Fit**: alokasi besar berulang
- **Next-Fit**: sistem dinamis dengan alokasi sering

Visualisasi (Penjelasan Laporan)

Visualisasi dilakukan dengan menampilkan status memori setelah setiap alokasi dan dealokasi, menunjukkan:

- Address awal
- Ukuran block
- Status allocated/free

TUGAS 3: ADVANCED PAGING SYSTEM

```
#include <stdio.h>

#include <stdlib.h>


#define FRAME_SIZE 256
#define NUM_FRAMES 4
#define MAX_PAGES 100


/* ===== PILIH ALGORITMA ===== */
// Uncomment SALAH SATU
#define USE_FIFO
// #define USE_LRU
/* ===== */


typedef struct {
    int page_number;
    int frame_number;
    int valid;
    int reference_bit;
    int modify_bit;
    int last_used;    // untuk LRU
} PageTableEntry;


typedef struct {
    int occupied;
    int page_number;
} Frame;


typedef struct {
    int total_accesses;
```

```

    int page_faults;
    int page_hits;
    float hit_ratio;
    int disk_writes;
} PageStatistics;

/* ===== GLOBAL ===== */

Frame physical_memory[NUM_FRAMES];
PageTableEntry page_table[MAX_PAGES];
PageStatistics stats;

int fifo_pointer = 0;
int current_time = 0;

/* ===== */

void initialize_system() {
    for (int i = 0; i < NUM_FRAMES; i++) {
        physical_memory[i].occupied = 0;
        physical_memory[i].page_number = -1;
    }

    for (int i = 0; i < MAX_PAGES; i++) {
        page_table[i].page_number = i;
        page_table[i].frame_number = -1;
        page_table[i].valid = 0;
        page_table[i].reference_bit = 0;
        page_table[i].modify_bit = 0;
        page_table[i].last_used = 0;
    }
}

```

```

stats.total_accesses = 0;
stats.page_faults = 0;
stats.page_hits = 0;
stats.disk_writes = 0;

fifo_pointer = 0;
current_time = 0;

printf("System initialized (Frames = %d)\n\n", NUM_FRAMES);
}

int find_free_frame() {
    for (int i = 0; i < NUM_FRAMES; i++) {
        if (!physical_memory[i].occupied)
            return i;
    }
    return -1;
}

/* ===== FIFO ===== */
int fifo_page_replacement(int new_page) {
    int frame = fifo_pointer;
    int old_page = physical_memory[frame].page_number;

    if (page_table[old_page].modify_bit) {
        stats.disk_writes++;
        printf("Disk write (FIFO): Page %d\n", old_page);
    }

    page_table[old_page].valid = 0;

```

```

page_table[old_page].frame_number = -1;
page_table[old_page].modify_bit = 0;

page_table[new_page].frame_number = frame;
page_table[new_page].valid = 1;
page_table[new_page].last_used = ++current_time;

physical_memory[frame].page_number = new_page;

fifo_pointer = (fifo_pointer + 1) % NUM_FRAMES;

return frame;
}
/* ===== */

/* ===== LRU ===== */
int lru_page_replacement(int new_page) {
    int lru_page = -1;
    int min_time = 1e9;

    for (int i = 0; i < MAX_PAGES; i++) {
        if (page_table[i].valid && page_table[i].last_used < min_time) {
            min_time = page_table[i].last_used;
            lru_page = i;
        }
    }

    int frame = page_table[lru_page].frame_number;

    if (page_table[lru_page].modify_bit) {

```

```

        stats.disk_writes++;

        printf("Disk write (LRU): Page %d\n", lru_page);
    }

    page_table[lru_page].valid = 0;
    page_table[lru_page].frame_number = -1;
    page_table[lru_page].modify_bit = 0;

    page_table[new_page].frame_number = frame;
    page_table[new_page].valid = 1;
    page_table[new_page].last_used = ++current_time;

    physical_memory[frame].page_number = new_page;

    return frame;
}

/* ===== */

void access_page(int page_num, int write) {
    stats.total_accesses++;

    if (page_table[page_num].valid) {
        stats.page_hits++;
        page_table[page_num].last_used = ++current_time;
        printf("Page %d HIT\n", page_num);
    } else {
        stats.page_faults++;
        printf("Page %d FAULT\n", page_num);

        int frame = find_free_frame();
    }
}

```

```

        if (frame == -1) {
#ifdef USE_FIFO
            frame = fifo_page_replacement(page_num);
#else
            frame = lru_page_replacement(page_num);
#endif
        } else {
            physical_memory[frame].occupied = 1;
            physical_memory[frame].page_number = page_num;

            page_table[page_num].frame_number = frame;
            page_table[page_num].valid = 1;
            page_table[page_num].last_used = ++current_time;
        }
    }

    if (write)
        page_table[page_num].modify_bit = 1;
}

void display_statistics() {
    stats.hit_ratio =
        (float)stats.page_hits / stats.total_accesses;

    printf("\n=== STATISTICS ===\n");
    printf("Total Accesses : %d\n", stats.total_accesses);
    printf("Page Faults   : %d\n", stats.page_faults);
    printf("Page Hits      : %d\n", stats.page_hits);
    printf("Hit Ratio      : %.2f\n", stats.hit_ratio);
}

```

```

    printf("Disk Writes   : %d\n", stats.disk_writes);
}

void run_test_scenario() {
    int reference_string[] = {
        7,0,1,2,0,3,0,4,2,3,
        0,3,2,1,2,0,1,7,0,1
    };

    int n = sizeof(reference_string) / sizeof(int);

    printf("\nRunning Test Scenario...\n\n");

    for (int i = 0; i < n; i++) {
        printf("Access %d → ", reference_string[i]);
        access_page(reference_string[i], 0);
    }

    display_statistics();
}

int main() {
    initialize_system();
    run_test_scenario();
    return 0;
}

```

- Output:

- USE FIFO

```
root@RIDH0:~# nano paging_simulator_final.c
root@RIDH0:~# gcc paging_simulator_final.c -o paging
root@RIDH0:~# ./paging
System initialized (Frames = 4)
```

Running Test Scenario...

```
Access 7 → Page 7 FAULT
Access 0 → Page 0 FAULT
Access 1 → Page 1 FAULT
Access 2 → Page 2 FAULT
Access 0 → Page 0 HIT
Access 3 → Page 3 FAULT
Access 0 → Page 0 HIT
Access 4 → Page 4 FAULT
Access 2 → Page 2 HIT
Access 3 → Page 3 HIT
Access 0 → Page 0 FAULT
Access 3 → Page 3 HIT
Access 2 → Page 2 HIT
Access 1 → Page 1 FAULT
Access 2 → Page 2 FAULT
Access 0 → Page 0 HIT
Access 1 → Page 1 HIT
Access 7 → Page 7 FAULT
Access 0 → Page 0 HIT
Access 1 → Page 1 HIT
```

=== STATISTICS ===

```
Total Accesses : 20
Page Faults     : 10
Page Hits       : 10
Hit Ratio        : 0.50
Disk Writes     : 0
```

- USE_LRU

```
root@RIDH0:~# nano paging_simulator_final.c
root@RIDH0:~# gcc paging_simulator_final.c -o paging
root@RIDH0:~# ./paging
System initialized (Frames = 4)
```

Running Test Scenario...

```
Access 7 → Page 7 FAULT
Access 0 → Page 0 FAULT
Access 1 → Page 1 FAULT
Access 2 → Page 2 FAULT
Access 0 → Page 0 HIT
Access 3 → Page 3 FAULT
Access 0 → Page 0 HIT
Access 4 → Page 4 FAULT
Access 2 → Page 2 HIT
Access 3 → Page 3 HIT
Access 0 → Page 0 HIT
Access 3 → Page 3 HIT
Access 2 → Page 2 HIT
Access 1 → Page 1 FAULT
Access 2 → Page 2 HIT
Access 0 → Page 0 HIT
Access 1 → Page 1 HIT
Access 7 → Page 7 FAULT
Access 0 → Page 0 HIT
Access 1 → Page 1 HIT
```

=== STATISTICS ===

```
Total Accesses : 20
Page Faults     : 8
Page Hits       : 12
Hit Ratio       : 0.60
Disk Writes     : 0
```

```
root@RIDH0:~# |
```

1. Perbandingan FIFO vs LRU

Algoritma Page Fault Hit Ratio Disk Write

FIFO Lebih banyak Lebih rendah Bisa lebih banyak

LRU Lebih sedikit Lebih tinggi Lebih efisien

2. Belady's Anomaly (FIFO)

Frames Page Fault

3 Lebih kecil

4 Lebih besar

5 Menurun

→ FIFO mengalami Belady's Anomaly

→ LRU TIDAK mengalami Belady's Anomaly

3. Kesimpulan Analisis

- LRU lebih optimal karena mempertimbangkan pola akses
- FIFO lebih sederhana, tapi bisa anomali
- FIFO masih layak untuk sistem kecil & sederhana
- LRU cocok untuk sistem modern

TUGAS 4: MULTI LEVEL PAGE TABLE DAN INVERTED TABLE

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
/* ===== CONFIG ===== */
```

```
#define L1_BITS 16
```

```
#define L2_BITS 8
```

```
#define L3_BITS 8
```

```
#define OFFSET_BITS 12
```

```
#define FRAME_SIZE (1 << OFFSET_BITS)
```

```
#define NUM_FRAMES 256 // demo
```

```
/* ===== STRUCTURES ===== */
```

```
typedef struct {
```

```
    int frame_number;
```

```
    int valid;
```

```
    int read;
```

```
    int write;
```

```
    int execute;
```

```
} L3_Entry;
```

```
typedef struct {
```

```
    L3_Entry *entries;
```

```
    int valid;
```

```
} L2_Entry;
```

```
typedef struct {
```

```

    L2_Entry *entries;

    int valid;
} L1_Entry;

/* ===== GLOBAL ===== */

L1_Entry *page_table_l1;

/* ===== INIT ===== */

void init_page_table() {
    page_table_l1 = calloc(1 << L1_BITS, sizeof(L1_Entry));
}

/* ===== ALLOCATE ===== */

void allocate_3level(unsigned long vaddr, int frame) {
    unsigned long l1 = (vaddr >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 <<
L1_BITS) - 1);
    unsigned long l2 = (vaddr >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
    unsigned long l3 = (vaddr >> OFFSET_BITS) & ((1 << L3_BITS) - 1);

    if (!page_table_l1[l1].valid) {
        page_table_l1[l1].entries = calloc(1 << L2_BITS, sizeof(L2_Entry));
        page_table_l1[l1].valid = 1;
    }

    if (!page_table_l1[l1].entries[l2].valid) {
        page_table_l1[l1].entries[l2].entries = calloc(1 << L3_BITS, sizeof(L3_Entry));
        page_table_l1[l1].entries[l2].valid = 1;
    }

    L3_Entry *entry = &page_table_l1[l1].entries[l2].entries[l3];
    entry->frame_number = frame;

```

```

    entry->valid = 1;
    entry->read = entry->write = entry->execute = 1;
}

/* ===== TRANSLATE ===== */
int translate_3level(unsigned long vaddr) {
    unsigned long l1 = (vaddr >> (L2_BITS + L3_BITS + OFFSET_BITS)) & ((1 <<
L1_BITS) - 1);
    unsigned long l2 = (vaddr >> (L3_BITS + OFFSET_BITS)) & ((1 << L2_BITS) - 1);
    unsigned long l3 = (vaddr >> OFFSET_BITS) & ((1 << L3_BITS) - 1);
    unsigned long offset = vaddr & ((1 << OFFSET_BITS) - 1);

    if (!page_table_l1[l1].valid) return -1;
    if (!page_table_l1[l1].entries[l2].valid) return -1;

    L3_Entry *entry = &page_table_l1[l1].entries[l2].entries[l3];
    if (!entry->valid) return -1;

    return entry->frame_number * FRAME_SIZE + offset;
}

/* ===== MEMORY OVERHEAD ===== */
void calculate_memory_overhead() {
    size_t total = sizeof(L1_Entry) * (1 << L1_BITS);
    printf("Memory Overhead (approx): %lu bytes\n", total);
}

/* ===== MAIN ===== */
int main() {
    init_page_table();

```

```

unsigned long vaddr = 0x123456789A;
allocate_3level(vaddr, 42);

int phys = translate_3level(vaddr);
if (phys != -1)
    printf("Virtual: %lx -> Physical: %d\n", vaddr, phys);
else
    printf("Page Fault!\n");

calculate_memory_overhead();
return 0;
}

```

- Output:

```

root@RIDHO:~# nano three_level_pagetable.c
root@RIDHO:~# gcc three_level_pagetable.c -o three
root@RIDHO:~# ./three
Virtual: 123456789a -> Physical: 174234
Memory Overhead (approx): 1048576 bytes

```

A. Three-Level Page Table

1. Konsep Dasar

Virtual address (demo 44 bit):

Bagian Bit

L1 index 16

L2 index 8

L3 index 8

Offset 12

Struktur hierarki:

L1 Page Table

└─ L2 Page Table

└─ L3 Page Table

└─ Frame fisik

Hanya page table yang dibutuhkan saja yang dialokasikan → **hemat memori** dibanding single-level.

2. Tahapan Pengerjaan Three-Level Page Table

Tahap 1 – Konfigurasi bit

```
#define L1_BITS 16
```

```
#define L2_BITS 8
```

```
#define L3_BITS 8
```

```
#define OFFSET_BITS 12
```

Tahap 2 – Struktur Page Table

- **L3** menyimpan frame fisik + permission
- **L2** menunjuk ke array L3
- **L1** menunjuk ke array L2

Tahap 3 – Translasi alamat

1. Pecah virtual address → L1, L2, L3, offset
2. Validasi setiap level
3. Gabungkan frame + offset → alamat fisik

Inverted Page Table

1. Konsep

- Satu entry per **frame fisik**
- Entry berisi:
 - process_id
 - page_number
- Pencarian → **linear / hashing**

Lebih hemat memori, tapi translasi lebih lambat.

2. Tahapan Pengerjaan

1. Definisikan struktur inverted table
2. Fungsi insert
3. Fungsi search (linear)

```
#include <stdio.h>
```

```
#define NUM_FRAMES 256
```

```
typedef struct {  
    int process_id;  
    int page_number;  
    int valid;  
} InvertedPageEntry;
```

```
InvertedPageEntry inverted_table[NUM_FRAMES];
```

```
/* ===== SEARCH ===== */
```

```
int search_inverted_table(int pid, int page_num) {  
    for (int i = 0; i < NUM_FRAMES; i++) {  
        if (inverted_table[i].valid &&  
            inverted_table[i].process_id == pid &&  
            inverted_table[i].page_number == page_num) {  
            return i; // frame number  
        }  
    }  
    return -1;  
}
```

```
/* ===== INSERT ===== */
```

```
void insert_inverted_table(int pid, int page_num, int frame) {  
    inverted_table[frame].process_id = pid;  
    inverted_table[frame].page_number = page_num;  
    inverted_table[frame].valid = 1;  
}
```

```
/* ===== MAIN ===== */  
  
int main() {  
    insert_inverted_table(1, 10, 5);  
  
    int frame = search_inverted_table(1, 10);  
    if (frame != -1)  
        printf("Page found in frame %d\n", frame);  
    else  
        printf("Page not found\n");  
  
    return 0;  
}
```

-Output:

```
root@RIDH0:~# nano inverted_table_page.c  
root@RIDH0:~# gcc inverted_table_page.c -o inverted  
root@RIDH0:~# ./inverted  
Page found in frame 5  
root@RIDH0:~# |
```

1. Memory Overhead

Struktur Overhead

Single-Level Sangat besar

Two-Level Lebih kecil

Three-Level Paling efisien (sparse)

Inverted Paling kecil

2. Translation Speed

Struktur Memory Access

Single-Level 1

Two-Level 2

Three-Level 3

Inverted N (linear)

- TLB sangat penting untuk multi-level
- Inverted table wajib hashing + TLB

3. Kasus Penggunaan

Sistem Cocok

32-bit Two-Level

64-bit Three-Level / Multi-Level

Memori terbatas Inverted

Performa tinggi Multi-Level + TLB

Kesimpulan

- Three-Level Page Table
 - ✓ Skalabel
 - ✓ Cocok sistem modern 64-bit
- Inverted Page Table
 - ✓ Hemat memori
 - ✗ Translasi lebih lambat

TUGAS 5: SWAPPING MECHANISM

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <string.h>


#define MAX_PROCESS 10

#define MEMORY_SIZE 1024

#define SIM_TIME 100


typedef struct {
    int pid;
    int size;
    int priority;
    int in_memory;
    int swap_count;
    time_t last_access;
    time_t load_time;
} Process;


/* Global */

Process processes[MAX_PROCESS];

int used_memory = 0;


/* Utility */

int random_range(int min, int max) {
    return min + rand() % (max - min + 1);
}


/* Swap Out */
```

```

void swap_out(Process *p) {
    if (p->in_memory) {
        p->in_memory = 0;
        used_memory -= p->size;
        p->swap_count++;
        printf("SWAP OUT: P%d (%d MB)\n", p->pid, p->size);
    }
}

```

/* Swap In */

```

void swap_in(Process *p, time_t current_time) {
    while (used_memory + p->size > MEMORY_SIZE) {
        // victim will be selected by strategy
        break;
    }
    p->in_memory = 1;
    p->load_time = current_time;
    p->last_access = current_time;
    used_memory += p->size;
    printf("SWAP IN : P%d (%d MB)\n", p->pid, p->size);
}

```

/* FIFO Strategy */

```

int select_victim_fifo() {
    time_t oldest = time(NULL);
    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].load_time <= oldest) {
            oldest = processes[i].load_time;

```

```

        victim = i;
    }
}
return victim;
}

```

/* LRU Strategy */

```

int select_victim_lru() {
    time_t oldest = time(NULL);
    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].last_access <= oldest) {
            oldest = processes[i].last_access;
            victim = i;
        }
    }
    return victim;
}

```

/* Priority Strategy */

```

int select_victim_priority() {
    int lowest_priority = 999;
    int victim = -1;

    for (int i = 0; i < MAX_PROCESS; i++) {
        if (processes[i].in_memory && processes[i].priority < lowest_priority) {
            lowest_priority = processes[i].priority;
            victim = i;
        }
    }
}

```

```

    }
    return victim;
}

/* Simulation */
void simulate(int strategy) {
    used_memory = 0;

    for (int i = 0; i < MAX_PROCESS; i++) {
        processes[i].in_memory = 0;
        processes[i].swap_count = 0;
    }

    printf("\n=== SIMULATION START ===\n");

    for (int t = 0; t < SIM_TIME; t++) {
        int p = rand() % MAX_PROCESS;
        time_t now = time(NULL);

        if (!processes[p].in_memory) {
            while (used_memory + processes[p].size > MEMORY_SIZE) {
                int victim;
                if (strategy == 0)
                    victim = select_victim_fifo();
                else if (strategy == 1)
                    victim = select_victim_lru();
                else
                    victim = select_victim_priority();

                if (victim == -1) break;
            }
        }
    }
}

```

```

        swap_out(&processes[victim]);
    }
    swap_in(&processes[p], now);
} else {
    processes[p].last_access = now;
}
}

printf("=== SIMULATION END ===\n");
}

/* Main */
int main() {
    srand(time(NULL));

    for (int i = 0; i < MAX_PROCESS; i++) {
        processes[i].pid = i;
        processes[i].size = random_range(50, 200);
        processes[i].priority = random_range(1, 10);
        processes[i].in_memory = 0;
        processes[i].swap_count = 0;
    }

    printf("\n--- FIFO STRATEGY ---\n");
    simulate(0);

    printf("\n--- LRU STRATEGY ---\n");
    simulate(1);

    printf("\n--- PRIORITY STRATEGY ---\n");

```

```
simulate(2);
```

```
printf("\n=== SUMMARY ===\n");
```

```
for (int i = 0; i < MAX_PROCESS; i++) {
```

```
    printf("P%d | Swap Count: %d\n", processes[i].pid, processes[i].swap_count);
```

```
}
```

```
return 0;
```

```
}
```

-output:

```
root@RIDHO:~# nano swapping_simulation.c
root@RIDHO:~# gcc swapping_simulation.c -o swapping
root@RIDHO:~# ./swapping
```

```
--- FIFO STRATEGY ---
```

```
=== SIMULATION START ===
```

```
SWAP IN : P1 (141 MB)
SWAP IN : P0 (51 MB)
SWAP IN : P2 (124 MB)
SWAP IN : P9 (64 MB)
SWAP IN : P6 (138 MB)
SWAP IN : P5 (127 MB)
SWAP IN : P3 (149 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P4 (76 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P7 (186 MB)
SWAP OUT: P7 (186 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P7 (186 MB)
SWAP OUT: P7 (186 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P7 (186 MB)
SWAP OUT: P7 (186 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P7 (186 MB)
SWAP OUT: P7 (186 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
```

```
SWAP OUT: P8 (170 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P7 (186 MB)
=== SIMULATION END ===
```

--- LRU STRATEGY ---

```
=== SIMULATION START ===
SWAP IN : P2 (124 MB)
SWAP IN : P9 (64 MB)
SWAP IN : P1 (141 MB)
SWAP IN : P5 (127 MB)
SWAP IN : P7 (186 MB)
SWAP IN : P3 (149 MB)
SWAP IN : P4 (76 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP IN : P0 (51 MB)
SWAP OUT: P8 (170 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P8 (170 MB)
```

--- PRIORITY STRATEGY ---

=== SIMULATION START ===

SWAP IN : P4 (76 MB)
SWAP IN : P6 (138 MB)
SWAP IN : P9 (64 MB)
SWAP IN : P8 (170 MB)
SWAP IN : P7 (186 MB)
SWAP IN : P5 (127 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P1 (141 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P0 (51 MB)
SWAP OUT: P0 (51 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P2 (124 MB)
SWAP IN : P0 (51 MB)
SWAP OUT: P0 (51 MB)
SWAP OUT: P5 (127 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P5 (127 MB)
SWAP OUT: P5 (127 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P5 (127 MB)
SWAP OUT: P5 (127 MB)
SWAP IN : P9 (64 MB)
SWAP IN : P0 (51 MB)
SWAP OUT: P9 (64 MB)
SWAP OUT: P0 (51 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P0 (51 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP OUT: P0 (51 MB)
SWAP IN : P3 (149 MB)

```
SWAP OUT: P9 (64 MB)
SWAP OUT: P0 (51 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P0 (51 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP OUT: P0 (51 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P0 (51 MB)
SWAP OUT: P0 (51 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P0 (51 MB)
SWAP IN : P5 (127 MB)
SWAP OUT: P0 (51 MB)
SWAP OUT: P5 (127 MB)
SWAP IN : P3 (149 MB)
SWAP OUT: P3 (149 MB)
SWAP IN : P9 (64 MB)
SWAP OUT: P9 (64 MB)
SWAP IN : P3 (149 MB)
=== SIMULATION END ===
```

```
=== SUMMARY ===
```

```
P0 | Swap Count: 6
P1 | Swap Count: 0
P2 | Swap Count: 0
P3 | Swap Count: 7
P4 | Swap Count: 0
P5 | Swap Count: 4
P6 | Swap Count: 0
P7 | Swap Count: 0
P8 | Swap Count: 0
P9 | Swap Count: 7
root@RIDH0:~# |
```

PENJELASAN KODE

A. Struktur Process

```
typedef struct {  
    int pid;  
    int size;  
    int priority;  
    int in_memory;  
    int swap_count;  
    time_t last_access;  
    time_t load_time;  
} Process;
```

Fungsi tiap field:

- pid → ID proses
- size → ukuran memory (MB)
- priority → prioritas proses
- in_memory → status (0 = swap, 1 = RAM)
- swap_count → jumlah swap
- last_access → untuk **LRU**
- load_time → untuk **FIFO**

B. Mekanisme Swap

Swap Out

```
void swap_out(Process *p)
```

- Mengeluarkan proses dari RAM
- Mengurangi used_memory
- Menambah swap_count

Swap In

```
void swap_in(Process *p)
```

- Memasukkan proses ke RAM
- Update waktu akses dan load

C. Strategi Swap

1. FIFO

- Proses **paling lama berada di memory** dikeluarkan dulu

select_victim_fifo()

2. LRU

- Proses **paling lama tidak diakses** dikeluarkan

select_victim_lru()

3. Priority-based

- Proses dengan **prioritas terendah** dikeluarkan

select_victim_priority()

D. Simulasi

simulate(strategy)

- Akses proses secara random selama **100 time units**
- Jika memory tidak cukup → pilih victim → swap out
- Lalu swap in proses baru

3. CONTOH ANALISIS (UNTUK LAPORAN)

Tabel Performa (Contoh)

Strategy Swap Out Count Swap In Count Total Swap Time Avg Response Time

FIFO	Tinggi	Tinggi	Tinggi	Lambat
LRU	Rendah	Rendah	Rendah	Cepat
Priority	Sedang	Sedang	Sedang	Sedang

Jawaban Analisis

1. Strategi yang meminimalkan thrashing

- **LRU**, karena mempertahankan proses yang sering diakses.

2. Pengaruh ukuran memory

- Memory kecil → swap sering → thrashing
- Memory besar → swap jarang → performa naik

3. Kapan swapping lebih baik daripada killing process?

- Proses penting / jangka panjang
- Sistem multitasking
- Ingin mempertahankan state proses

TUGAS 6: ARCHITECTURE ANALYSIS

A. Intel Architecture analysis

1. Cek informasi cpu:

```
root@RIDHO:~# lscpu | grep -E "Architecture|CPU|Model|Thread|Core"
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
CPU(s):                20
On-line CPU(s) list:   0-19
Thread(s) per core:    2
Core(s) per socket:    10
CPU family:            6
Model:                 154
Model name:            12th Gen Intel(R) Core(TM) i7-12700H
CPU MHz:               2688.000
root@RIDHO:~# |
```

Analisis:

- Sistem menggunakan **arsitektur 64-bit (x86_64)**.
- Mendukung **multithreading (SMT / Hyper-Threading)**.
- Jumlah core fisik dan logical core menunjukkan dukungan paralelisme yang baik.

2. Cek page size:

```
root@RIDHO:~# getconf PAGE_SIZE
4096
root@RIDHO:~# |
```

Analisis:

- Ukuran page adalah **4 KB**.
- Ini adalah ukuran page standar pada Linux modern.
- Digunakan oleh OS untuk **virtual memory management** dan paging.

3. Cek TLB Info:

```
root@RIDHO:~# cpuid | grep -i tlb

Command 'cpuid' not found, but can be installed with:
apt install cpuid

root@RIDHO:~# |
```

4. Cek hierarchy cache

```
root@RIDHO:~# lscpu | grep -i cache
L1d cache: 480 KiB
L1i cache: 320 KiB
L2 cache: 12.5 MiB
L3 cache: 24 MiB
root@RIDHO:~# |
```

Analisis:

- L1: cache tercepat, dekat dengan core.
- L2: cache menengah.
- L3: shared antar core.
- Hirarki cache mengurangi latency akses memori utama.

B. Memory Information

1. Total Memory:

```
root@RIDHO:~# free -h
              total        used        free      shared  buff/cache   available
Mem:           7.6Gi        141Mi        7.4Gi          0.0Ki         95Mi        7.3Gi
Swap:          2.0Gi           0B          2.0Gi
```

2. Virtual memory info

```
root@RIDHO:~# vmstat 1 5
procs -----memory----- --swap--  -----io----- -system--  -----cpu-----
 r  b    swpd    free    buff  cache   si   so    bi   bo    in   cs us sy id wa st
 0  0      0  7708420  13060  85048    0    0     0    20    0    1  0  0 100  0  0
 0  0      0  7708420  13060  85048    0    0     0     0    3   24  0  0 100  0  0
 0  0      0  7708420  13060  85088    0    0     0     0    9   32  0  0 100  0  0
 0  0      0  7708420  13060  85088    0    0     0     0    4   28  0  0 100  0  0
 0  0      0  7708420  13060  85088    0    0     0     0    4   28  0  0 100  0  0
root@RIDHO:~# |
```

3. Memory map dari proses

```
root@RIDHO:~# vmstat 1 5
procs -----memory----- --swap-- ----io---- -system-- -----cpu-----
 r  b   swpd   free   buff   cache   si   so   bi   bo   in   cs   us   sy   id   wa   st
 0   0       0 7708420 13060 85048    0    0    0   20    0    1    0    0 100    0    0
 0   0       0 7708420 13060 85048    0    0    0    0    3   24    0    0 100    0    0
 0   0       0 7708420 13060 85088    0    0    0    0    9   32    0    0 100    0    0
 0   0       0 7708420 13060 85088    0    0    0    0    4   28    0    0 100    0    0
 0   0       0 7708420 13060 85088    0    0    0    0    4   28    0    0 100    0    0

root@RIDHO:~# cat /proc/self/maps
56309ada1000-56309ada3000 r--p 00000000 08:10 1773                /usr/bin/cat
56309ada3000-56309ada8000 r-xp 00002000 08:10 1773                /usr/bin/cat
56309ada8000-56309adab000 r--p 00007000 08:10 1773                /usr/bin/cat
56309adab000-56309adac000 r--p 00009000 08:10 1773                /usr/bin/cat
56309adac000-56309adad000 rw-p 0000a000 08:10 1773                /usr/bin/cat
56309bf21000-56309bf42000 rw-p 00000000 00:00 0                [heap]
7fd0bc53c000-7fd0bc55e000 rw-p 00000000 00:00 0
7fd0bc55e000-7fd0bc590000 r--p 00000000 08:10 22158            /usr/lib/locale/C.UTF-8/LC_CTYPE
7fd0bc590000-7fd0bc591000 r--p 00000000 08:10 22164            /usr/lib/locale/C.UTF-8/LC_NUMERIC
7fd0bc591000-7fd0bc592000 r--p 00000000 08:10 22167            /usr/lib/locale/C.UTF-8/LC_TIME
7fd0bc592000-7fd0bc705000 r--p 00000000 08:10 22157            /usr/lib/locale/C.UTF-8/LC_COLLATE
7fd0bc705000-7fd0bc706000 r--p 00000000 08:10 22162            /usr/lib/locale/C.UTF-8/LC_MONETARY
7fd0bc706000-7fd0bc707000 r--p 00000000 08:10 22161            /usr/lib/locale/C.UTF-8/LC_MESSAGES/SYS_LC_MESSAGES
7fd0bc707000-7fd0bc708000 r--p 00000000 08:10 22165            /usr/lib/locale/C.UTF-8/LC_PAPER
7fd0bc708000-7fd0bc709000 r--p 00000000 08:10 22163            /usr/lib/locale/C.UTF-8/LC_NAME
7fd0bc709000-7fd0bc70a000 r--p 00000000 08:10 22156            /usr/lib/locale/C.UTF-8/LC_ADDRESS
7fd0bc70a000-7fd0bc9f0000 r--p 00000000 08:10 4128             /usr/lib/locale/locale-archive
7fd0bc9f0000-7fd0bca12000 r--p 00000000 08:10 41732            /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fd0bca12000-7fd0bcb8a000 r-xp 00022000 08:10 41732            /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fd0bcb8a000-7fd0bcbdb000 r--p 0019a000 08:10 41732            /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fd0bcbdb000-7fd0bcbdc000 r--p 001e7000 08:10 41732            /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fd0bcbdc000-7fd0bcbde000 rw-p 001eb000 08:10 41732            /usr/lib/x86_64-linux-gnu/libc-2.31.so
7fd0bcbde000-7fd0bcbef000 rw-p 00000000 00:00 0
7fd0bcbef000-7fd0bcbf0000 r--p 00000000 08:10 22166            /usr/lib/locale/C.UTF-8/LC_TELEPHONE
7fd0bcbf0000-7fd0bcbf1000 r--p 00000000 08:10 22160            /usr/lib/locale/C.UTF-8/LC_MEASUREMENT
7fd0bcbf1000-7fd0bcbf2000 r-xp 00000000 08:10 41999            /usr/lib/x86_64-linux-gnu/gconv/gconv-modules.cache
7fd0bcbf2000-7fd0bcbf3000 r--p 00000000 08:10 40200            /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fd0bcbf3000-7fd0bcbf4000 r-xp 00001000 08:10 40200            /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fd0bcbf4000-7fd0bcbf5000 r--p 00024000 08:10 40200            /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fd0bcbf5000-7fd0bcbf6000 r--p 00000000 08:10 22159            /usr/lib/locale/C.UTF-8/LC_IDENTIFICATION
7fd0bcbf6000-7fd0bcbf7000 r--p 0002c000 08:10 40200            /usr/lib/x86_64-linux-gnu/ld-2.31.so
7fd0bcbf7000-7fd0bcbf8000 rw-p 0002d000 08:10 40200            /usr/lib/x86_64-linux-gnu/ld-2.31.so
```

```
7fd0bcc1b000-7fd0bcc1c000 r--p 0002a000 00:00 0
7fd0bcc1c000-7fd0bcc1d000 rw-p 00000000 00:00 0
7ffe1f1d9000-7ffe1f1fa000 rw-p 00000000 00:00 0                [stack]
7ffe1f1fa000-7ffe1f1fe000 r--p 00000000 00:00 0                [vvar]
7ffe1f1fe000-7ffe1f1ff000 r-xp 00000000 00:00 0                [vdso]
root@RIDHO:~# |
```

Analisis:

- Menunjukkan segment:
 - text
 - heap
 - stack
 - shared libraries
- Membuktikan penggunaan **virtual address space**.

4. Swap usage

```
root@RIDHO:~# swapon --show
NAME      TYPE  SIZE USED PRIO
/swap/file file   2G   0B  -2
root@RIDHO:~# |
```

Analisis:

- Swap aktif membantu saat RAM penuh.
- Terlalu sering swap → indikasi kekurangan RAM.

C. Analisis (Dalam Laporan)

1. Perbandingan Arsitektur

Arsitektur	Address Space	Maks. Memori
Intel 32-bit (IA-32)	2^{32}	4 GB
Intel 64-bit (x86-64)	2^{64} (teoretis)	TB–PB
ARMv8	64-bit	Serupa x86-64

2. Jawaban Analisis

a. Berapa bit address space yang digunakan?

- Sistem menggunakan **64-bit address space**.
- Virtual address Linux umumnya **48 bit efektif**.

b. Bagaimana implementasi multi-level paging?

- x86-64 menggunakan **4–5 level page table**:
 - PGD → PUD → PMD → PTE
- Mengurangi penggunaan memori page table.

c. Berapa ukuran maksimum virtual memory?

- Secara teori: **2^{64} byte**
- Praktis (Linux): ratusan TB per proses.

KESIMPULAN

- Sistem menggunakan arsitektur **64-bit modern**.
- Mendukung virtual memory, TLB, dan cache multi-level.
- Desain ini meningkatkan performa dan skalabilitas sistem.

KESIMPULAN

Main memory merupakan komponen vital dalam sistem komputer yang berfungsi sebagai penghubung utama antara prosesor dan penyimpanan sekunder. Analisis memory layout membantu memahami bagaimana ruang alamat dibagi untuk kode, data, heap, dan stack sehingga pengelolaan memori dapat dilakukan secara efisien dan aman.

Implementasi memory allocator berperan penting dalam mengatur alokasi dan dealokasi memori secara dinamis. Algoritma allocator yang baik mampu meminimalkan fragmentasi serta meningkatkan performa aplikasi. Pada tingkat yang lebih lanjut, advanced paging system memungkinkan sistem operasi mengelola memori virtual secara lebih fleksibel dan efisien, terutama dalam mendukung proses multitasking.

Penggunaan multi-level page table terbukti efektif dalam menghemat penggunaan memori untuk page table itu sendiri, terutama pada sistem dengan ruang alamat yang besar. Mekanisme ini memungkinkan skalabilitas sistem tanpa membebani memori fisik secara berlebihan.

Selain itu, swapping mechanism memberikan solusi ketika memori fisik tidak mencukupi dengan memindahkan halaman memori yang jarang digunakan ke penyimpanan sekunder. Meskipun membantu menjaga stabilitas sistem, swapping yang berlebihan dapat menurunkan kinerja secara signifikan.

Terakhir, analisis arsitektur memori menunjukkan bahwa desain dan organisasi memori sangat memengaruhi performa sistem secara keseluruhan. Arsitektur yang tepat dapat mengoptimalkan akses memori, mengurangi latensi, dan meningkatkan efisiensi eksekusi program.

SARAN

Dalam penerapan dan pembelajaran materi main memory, pemahaman terhadap konsep memory layout dan memory allocator sebaiknya tidak hanya bersifat teoritis, tetapi juga didukung dengan praktik langsung agar pengelolaan memori dapat dilakukan secara efisien dan aman. Implementasi sistem paging lanjutan serta multi-level page table perlu disesuaikan dengan kebutuhan sistem dan kemampuan perangkat keras untuk menghindari overhead yang dapat menurunkan kinerja. Mekanisme swapping harus diatur secara bijak agar tidak terlalu sering terjadi pemindahan data antara memori utama dan penyimpanan sekunder yang dapat menyebabkan penurunan performa sistem. Selain itu, perancangan dan analisis arsitektur memori sebaiknya dilakukan sejak awal pengembangan sistem agar akses memori lebih optimal, latensi dapat diminimalkan, serta stabilitas sistem tetap terjaga. Dengan terus berkembangnya teknologi komputer, pembelajaran dan penelitian lanjutan mengenai optimasi manajemen memori sangat disarankan guna menghasilkan sistem yang lebih efisien dan andal.